BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS
ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

GCC PLUGIN FOR STATIC ANALYZER SUPPORT

GCC PLUGIN PRO STATICKÉ ANALYZÁTORY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **LUKÁŠ PŠEJA**
AUTOR PRÁCE

SUPERVISOR **Dr. Ing. PETR PERINGER,**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

172017

Institut:	Department of Intelligent Systems (DITS)
Student:	**Pšeja Lukáš**
Programme:	Information Technology
Title:	**GCC plugin for static analyzer support**
Category:	Software analysis and testing
Academic year:	2025/26

Assignment:

1. Get acquainted with *CodeListener* component from the *Predator* project. Analyze the implementation of *CodeListener* and similar projects. Additionally, study the relevant data structure visualization concepts.
2. Design a new GCC plugin as a modern alternative to *CodeListener* and add new functionality (built-in visualization support and export in JSON format). Focus on improvement of the plugin interface.
3. Implement the plugin in C++ along with appropriate examples for demonstration purposes. Verify the plugin's compatibility by using it with *Predator*.
4. Test the plugin using the created examples with various GCC versions. Evaluate obtained results.

Literature:

- GitHub repository of Predator project. https://github.com/kdudka/predator

Requirements for the semestral defence:

- First two points of assignment.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Peringer Petr, Dr. Ing.**
Head of Department:	Kočí Radek, Ing., Ph.D.
Beginning of work:	1.11.2025
Submission deadline:	13.5.2026
Approval date:	3.11.2025

Abstract

This bachelor's thesis modernizes the Code Listener infrastructure, which serves as an integration layer between compilers and static analyzers for C and C++ programs. It first analyzes the original architecture and identifies its main limitations, namely mandatory transformations and the pollution of the internal program representation with analysis results. Based on this analysis, the thesis designs and implements a new GCC plugin built around a compiler-independent CodeModel, lazily evaluated annotation services, a unified analyzer interface, and JSON export for offline processing. The solution preserves backward compatibility with Predator through an adapter that reconstructs the original callback stream from the new CodeModel. Correctness and practical viability were validated by native regression suites and by 1,192 Predator compatibility tests executed after rebuilds with GCC 12, 13, and 14.

Abstrakt

Tato bakalářská práce se zabývá modernizací infrastruktury Code Listener, která slouží jako mezivrstva mezi překladači a statickými analyzátory programů v jazyce C a C++. Nejprve analyzuje původní architekturu a identifikuje její hlavní omezení, zejména povinné transformace a propletení výsledků analýz s vnitřní reprezentací programu. Na základě této analýzy navrhuje a implementuje nový plugin pro GCC postavený na překladači nezávislém modelu programu CodeModel, líně vyhodnocovaných anotačních službách, jednotném rozhraní pro analyzátory a podpoře exportu do JSON pro offline zpracování. Řešení zachovává zpětnou kompatibilitu s nástrojem Predator prostřednictvím adaptéru, který rekonstruuje původní proud zpětných volání z nového CodeModelu. Správnost a praktická použitelnost návrhu byly ověřeny nativními regresními testy a 1192 testy kompatibility Predatoru při sestaveních s GCC 12, 13 a 14.

Keywords

Code Listener, GCC plugin, static analysis, CodeModel, JSON serialization, Predator, backward compatibility

Klíčová slova

Code Listener, GCC plugin, statická analýza, CodeModel, serializace JSON, Predator, zpětná kompatibilita

Reference

PŠEJA, Lukáš. *GCC plugin for static analyzer support.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Dr. Ing. Petr Peringer,

GCC plugin for static analyzer support

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Dr. Ing. Petr Peringer. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Grammarly to improve the linguistic quality and spelling of this thesis. I utilized GitHub Copilot when working on the software. Specifically during the software development phase, where this tool was used exclusively for reviewing, consulting, and verifying my original interface designs and implementation choices, rather than for the automated code generation.

.....

Lukáš Pšeja

April 28, 2026

Contents

List of Figures

Chapter 1

Introduction

This bachelor's thesis focuses on the design and implementation of a modernized reimplementation of the Code Listener infrastructure, a C++ library that serves as a vital bridge between compilers and static analysis tools, particularly the Predator shape analysis tool suite. The original Code Listener architecture, while successful in enabling static analysis of C programs that manipulate heap memory, exhibits several design limitations that have accumulated over time. This thesis aims to address these constraints by replacing the callback-driven pipeline with a persistent Code Model, demoting analyses from hard-wired pre-analysis steps to optional, on-demand annotation services, and introducing a structured query interface and uniform support for exporting results in common formats.

1.1 Motivation

Software development is a complex and error-prone process, especially as software systems grow in size and complexity. As software systems evolve, certain categories of defects become undetectable solely through testing, as test suites can only evaluate explicitly covered execution paths, leaving much of the program's state space unexplored.

Nowhere is this limitation more critical than in low-level system programming languages such as C and C++, where developers bear the burden of manual memory management. In these environments, insidious bugs such as memory leaks, buffer overflows, use-after-free errors, and dangling pointer dereferences frequently hide within unexplored execution paths. The consequences of these hidden defects can be catastrophic, particularly in mission-critical domains like aerospace engineering. While rigorous testing prevents many fatal accidents, memory mismanagement has repeatedly compromised modern aircraft. For instance, regulatory agencies have had to issue mandatory airworthiness directives for both the Boeing 787 and the Airbus A350, requiring airlines to periodically reboot the aircraft to clear memory buffers and prevent the total mid-flight loss of critical systems. These near-misses underscore a fundamental reality: traditional dynamic testing is insufficient to guarantee memory safety in high-stakes environments.

To prevent such failures, developers must rely on advanced static analysis tools, such as the Predator shape analysis suite, which is capable of formally verifying a program's state space without executing it. However, developing and maintaining these analyzers presents a significant architectural challenge: extracting the program's semantics from the compiler's internal representation. Analyzers must interface with compilers to understand the code, but native compiler APIs are notoriously complex, tightly coupled to specific versions, and

largely undocumented. Building analysis tools directly on top of these raw APIs leads to brittle, maintenance-heavy software.

To overcome this bottleneck, a clean abstraction layer is required. Specifically, researchers need a modular middleware that can abstract away the massive, often undocumented codebases of industrial compilers, providing instead a concise, unified, and well-documented object-oriented API. This ensures that static analysis tools can be developed independently of the underlying compiler architecture.

1.2 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides a deeper technical analysis of the legacy Code Listener architecture. Chapter 3 outlines the architectural modernization, detailing the transition from a callback-driven pipeline to a persistent Code Model. Chapter 4 documents the implementation process, including development environment setup, portability considerations, and code metrics regarding the refactored infrastructure. Chapter 5 presents the methodology and results of regression testing, demonstrating the correctness and performance of the new implementation against the existing test suite. Finally, Chapter 6 summarizes the contributions of this work and outlines potential avenues for future research and development.

Chapter 2

State of the Art

This chapter provides a detailed account of the existing Code Listener infrastructure in the Predator GitHub repository [1] and as described in the original publication [2]. Understanding the design decisions and limitations of the current implementation is necessary background for the design choices made in this thesis.

To provide this context, the chapter first introduces the Code Listener infrastructure and compares it to similar projects within the broader static analysis ecosystem in Section 2.1 on page 6. It then details the underlying GCC compiler architecture and its GIMPLE intermediate representation, from which the program semantics are initially extracted in Section 2.2 on page 9. Following this, the internal architecture of the Code Listener and the importance of data structure visualization are examined in Section 2.3 on page 9. Finally, the chapter concludes with Section 2.4 on page 11 by identifying the critical limitations of the legacy implementation, directly motivating the architectural modernization proposed in the subsequent chapter.

2.1 The Code Listener Infrastructure

The Code Listener (CL) infrastructure is an open-source middleware designed to simplify the construction of static analysis tools for C [2] and, by extension, C++ [6] programs. It was developed by the VeriFIT research group [2] at Brno University of Technology mainly in the years 2009–2015, with some contributors continuing work on it until the present [3]. Its primary goal is to wrap the complex and often undocumented interfaces of existing code parsers to provide a unified, well-documented, and object-oriented Application Programming Interface (API) for static analysis tool developers [2]. By transforming the native internal representations of compilers into its own simplified format, the Code Listener API exposed to static analyzers becomes completely independent of the underlying parser. This abstraction layer allows the same analysis tool to run across different compilers without requiring modifications to the analyzer's source code [2].

The architecture of the original Code Listener is highly modular, effectively acting as a bridge between the compiler's internal representations and the static analysis tools, as illustrated in Figure 2.1 on page 7, and is composed of several key components:

[1] https://github.com/kdudka/predator
[2] https://www.fit.vut.cz/research/group/verifit/.en
[3] https://github.com/kdudka/predator/graphs/contributors

- **Code Parser Interface** is a layer that communicates directly with the source code parsers, such as the GCC compiler. Small adapters embedded within the code parsers are responsible for translating the parser's specific intermediate representation into the parser-independent representation required by Code Listener.
- **Filters and Listeners** are components that process intermediate code once it is obtained through the parser interface. Filters perform various transformations on the intermediate code, for example, breaking down a complex `SWITCH` instruction into a series of simpler `COND` instructions. Listeners, on the other hand, observe the stream of events generated by the code parser interface and can be used to provide diagnostic information or to generate visualizations, such as control flow graph (CFG) plotters, or intermediate code printers.
- **Code Storage** aggregates the incoming data to build a persistent, serialized object model of the analyzed source code, because advanced static analyzers often need to traverse the program structure repeatedly, which would be inefficient if they had to rely on a simple callback-driven pipeline. This component allows the analyzers to look up containers of types, variables, and functions, and to navigate the CFG of each function.
- **Analyzers** are the static analysis tools that can query the API and begin their formal verification operations once the code storage builds the complete object model. Any diagnostic messages emitted by the analyzers are passed back to the code parser interface, which formats them in the standard way expected by the compiler, including file names and line numbers, so that they can be easily understood by developers.

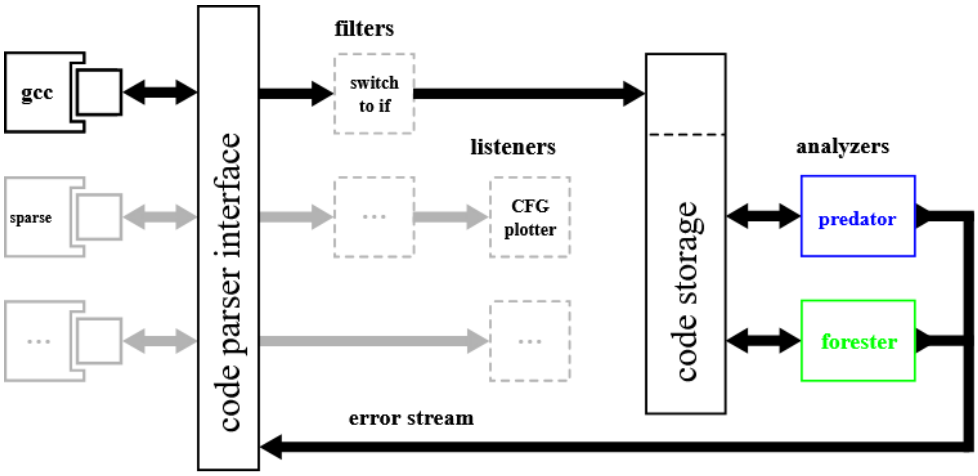


Figure 2.1: Block diagram of the legacy Code Listener infrastructure. The system is divided into interconnected modules, beginning with the code parser interface, which extracts intermediate representations from compilers such as GCC or Sparse. This data flows through optional filters, such as `switch to if` transformations, and listeners like the CFG plotters before being aggregated into a persistent object model within the code storage component. Finally, static analyzers such as Predator and Forester can query this code storage to perform formal verification, routing any discovered defects back through the code parser interface and the code parser's error stream to maintain consistent compiler-like error reporting [2].

The utility of the Code Listener infrastructure is demonstrated by the advanced verification tools that rely on it. The two primary analyzers built on top of Code Listener are Predator [12] and Forester [2]. Predator is a tool for automated formal verification of low-level C programs that manipulate dynamic, pointer-like data structures, such as linked lists [16]. Forester focuses on verifying programs that manipulate complex dynamic data structures using tree automata [15]. By using Code Listener as a compiler plugin, these tools can seamlessly analyze anything the compiler can process. This ensures that the user does not need to manually preprocess source code or modify standard build systems to run the analyzers [2], which is a significant advantage for usability and integration with existing development workflows.

Similar Projects and Ecosystem Comparisons

To contextualize the architectural choices of the Code Listener infrastructure, it is valuable to compare it against peer projects in the static analysis domain.

The primary peer to Code Listener is the Frama-C platform, an extensible, open-source framework dedicated to the static analysis of C source code [7]. Much like Code Listener, Frama-C isolates its analysis plug-ins from the raw compiler to provide a unified Application Programming Interface. It achieves this by using a modified version of the CIL (C Intermediate Language) front-end, which parses ISO C99 programs into a normalized Abstract Syntax Tree (AST) [7]. Frama-C utilizes a highly collaborative architecture where various plug-ins, such as those based on abstract interpretation or deductive verification, share this AST and communicate via ACSL (ANSI/ISO C Specification Language) annotations [7]. While Frama-C operates primarily at the AST level, Code Listener translates compiler internals into a reduced, instruction-based control flow graph. The key advantage of Code Listener's approach is that it allows building of analyzers capable of handling everything industrial compilers are able to compile [2], whereas Frama-C is limited to the subset of C that can be represented in its AST and supported by the CIL front-end [7].

Other notable comparisons can be drawn from the LLVM compiler infrastructure. The Clang Static Analyzer, built on top of the Clang/LLVM architecture, provides an extensible framework for checking C and C++ code, though historically its scope has been more limited to compilation and AST-level linting than Frama-C's exhaustive verification [7]. However, the LLVM core provides a highly optimized, typed, Static Single Assignment (SSA) intermediate representation [9]. Notably, modern LLVM utilizes an `AnalysisManager` approach that elegantly solves the "God Object" anti-pattern observed in the legacy Code Listener, which is described in Section 2.4 on page 12. Instead of polluting the core Intermediate Representation (IR) with analysis results, LLVM's `AnalysisManager` computes results lazily and stores them externally, keyed by the IR node identity [9]. This clean separation of concerns heavily inspires the modernized architecture proposed in this thesis.

Outside the C and C++ domain, the Soot framework provides a highly successful, conceptually similar infrastructure for the static analysis of Java programs [8]. Much like Code Listener acts as an abstraction layer over native compiler structures, Soot addresses the difficulty of directly analyzing stack-based Java Virtual Machine (JVM) bytecode by transforming it into a simplified, typed three-address intermediate representation known as Jimple [8]. By exposing local data flow along a Control Flow Graph (CFG), Soot isolates researchers from the complexities of the underlying bytecode, paralleling Code Listener's decoupling of analyzers from GCC [8].

2.2 The GCC Architecture and GIMPLE Representation

To understand how the Code Listener extracts program semantics, it is essential to examine the internal architecture of the GNU Compiler Collection (GCC). GCC operates as a sequential pipeline divided into three primary blocks: the front end, the middle end, and the back end, and uses three main intermediate languages to represent a program during compilation, specifically GENERIC, GIMPLE, and RTL [4].

The front end is responsible for the lexical analysis and parsing of a specific programming language. It generates GENERIC, a common, language-independent representation that can represent programs written in all languages supported by GCC [4]. GENERIC serves as the standardized interface between the parser and the optimizer [4].

The middle end optimizes the program using the GIMPLE representation [4]. The compiler pass responsible for converting GENERIC into GIMPLE is called the *gimplifier* [4]. At this stage, GCC also supports loadable modules called plugins, which are invoked at pre-determined locations in the compilation process to provide extra features to the compiler [4]. The Code Listener infrastructure utilizes this plugin API, registering callbacks for specific events to intercept the compilation process and access the intermediate code without modifying the compiler itself [2].

Finally, the back end uses RTL (Register Transfer Language) to perform architecture-dependent optimizations and generate the assembly code for the specific target hardware [4].

The GIMPLE Intermediate Language

When intercepting the compilation pipeline, the Code Listener adapter parses the GIMPLE representation [2]. GIMPLE is a tree-based representation derived from GENERIC, specifically designed to facilitate target- and language-independent optimizations [4].

A defining characteristic of GIMPLE is its highly restrictive grammar. Expressions are broken down into a three-address representation consisting of tuples containing no more than three operands, with the exception of function calls [4]. To accommodate the breakdown of complex expressions, the compiler introduces artificial temporary variables to hold intermediate values [4]. Furthermore, expressions with side effects are strictly limited to the right-hand side of assignments [4].

Additionally, GIMPLE drastically simplifies the program's control flow. All high-level control structures used in GENERIC are dismantled and lowered into conditional jumps [4]. Lexical scopes are removed entirely, and exception regions are extracted and converted into an on-the-side exception region tree [4].

By flattening source code into this concise, explicit-control-flow format, GIMPLE provides a highly predictable foundation. The Code Listener adapter systematically translates these GIMPLE trees into its own Code Listener IR (CL IR), utilizing a custom control flow graph to fully decouple the static analyzers from the compiler's internal complexities [2].

2.3 Code Listener Internal Architecture

To decouple static analysis tools from the GCC compiler, the Code Listener adapter systematically translates GIMPLE into its own simplified format, the Code Listener Intermediate Representation (CL IR). In this representation, the execution structure of each parsed function is modeled using a Control Flow Graph (CFG) [2].

A Control Flow Graph is a directed mathematical graph where the vertices represent basic blocks, and the edges represent the possible transfers of control flow during program execution [2]. A basic block is defined as a straight-line sequence of code with exactly one entry point and one exit point. Once execution enters a basic block, all instructions within it are executed sequentially, without branching or halting, until the end of the block is reached.

To construct these blocks, the Code Listener API restricts its intermediate instruction set to two distinct groups:

- **Non-terminal instructions** which form the body of a basic block and cannot be used to exit it [2]. The set is highly reduced, consisting only of unary operations (`UNOP`), binary operations (`BINOP`), and function calls (`CALL`) [2].
- **Terminal instructions** are strictly located at the very end of a basic block and are responsible for determining the outgoing edges of the CFG [2]. This group includes unconditional jumps (`JMP`), conditional jumps (`COND`), switch statements (`SWITCH`), function returns (`RET`), and program aborts (`ABORT`) [2].

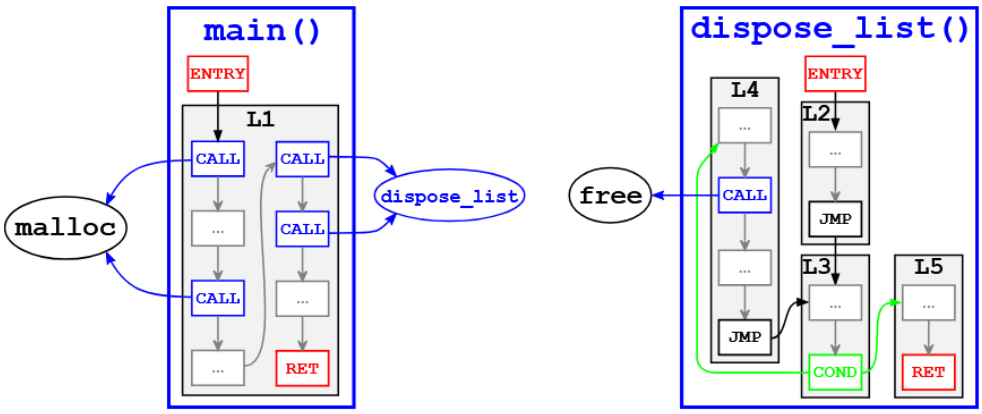


Figure 2.2: Control Flow Graph (CFG) representation of functions within the Code Listener Intermediate Representation. The graph models the execution structure using basic blocks as vertices and control flow transitions as directed edges. Each basic block contains a strictly sequential list of non-terminal instructions, such as function calls (`CALL`) or binary operations (`BINOP`). To determine the subsequent flow of execution, every block concludes with exactly one terminal instruction, such as an unconditional jump (`JMP`), conditional jump (`COND`), or return (`RET`) [2].

Instructions manipulate data through operands, represented by the `cl_operand` structure. An operand must refer either to a variable, such as a local or global variable, or a constant, such as a numeric literal, a string, or a function definition [2]. To alter the semantic meaning of an operand, for instance, to dereference a pointer, access an array element, or read a specific field within a composite structure, Code Listener uses accessors (`cl_accessor`) [2]. To guarantee simplicity for the analysis tools, Code Listener enforces a strict rule: the chaining of multiple dereferences is not allowed, and whenever a multiple dereference appears in the original C source code, the adapter automatically breaks it down into a sequence of separate instructions, ensuring each operand contains at most one dereference [2].

Furthermore, the C programming language is statically typed, and Code Listener preserves all type information known at compile-time. Every operand and accessor is explicitly associated with a type (`cl_type`), and composite types are defined recursively based on their constituent elements [2]. Because variable names in C source code are often ambiguous due to lexical scoping, Code Listener assigns a globally unique integer identifier (`uid`) to every variable and function [2].

Ultimately, the incoming stream of instructions, operands, and types is aggregated to build a persistent, serialized object model known as the Code Storage [2]. This C++ API provides the static analyzers with look-up databases for types (`TypeDb`), variables (`VarDb`), and functions (`FncDb`), effectively delivering a complete, parser-independent representation of the program ready for formal verification.

Data Structure Visualization Concepts

Static analysis tools extract and generate massive amounts of complex semantic data. To ensure that the Code Listener correctly translates the compiler's internal representation and to allow users to interpret analysis results, robust data-structure visualization is required.

Program Graph Visualization

The execution structure of a program is inherently non-linear and is best modeled mathematically as a directed graph. Consequently, visualizing the program's Control Flow Graph (CFG) and Call Graph is crucial for inspecting what the infrastructure has extracted. By outputting these structures into standard graph description languages, such as the DOT format, developers can generate visual plots of the basic blocks, terminal instructions, and branching paths [2]. This provides immediate visual confirmation that the adapter has successfully mapped the compiler's intermediate code without losing structural integrity.

Heap Shape Visualization

Beyond control flow, visualization is particularly vital for the specific domain of the Predator shape analyzer. Predator verifies programs that manipulate dynamically linked data structures by using an abstract interpretation domain based on Symbolic Memory Graphs (SMGs) [12]. Because tracking infinite sets of heap configurations, such as cyclic doubly-linked lists, is highly complex, Predator can produce error traces in a graphical format [12]. Visualizing these heap shapes enables analysts to graphically track memory leaks, invalid pointer dereferences, and the layout of the memory, bridging the gap between abstract mathematical models and human comprehension.

2.4 Limitations of the Legacy Architecture

While the legacy Code Listener infrastructure successfully abstracts the compiler's internal representation and has enabled tools like Predator to achieve high precision and win several awards [16], its architectural design exhibits several critical limitations. These constraints primarily stem from its inflexible filter pipeline, eager analysis execution, and tight coupling of analysis results to the core data structures.

Hardwired Filter Pipeline

The legacy GCC plugin configures its listener chains, such as diagnostic printers, graph plotters, and the analyzer itself, before the compilation process begins. Consequently, a predetermined set of intermediate filters, such as the `unfold_switch` filter, which converts `SWITCH` instructions into chains of conditional jumps, is hardwired for all listener invocations. This presents a binary choice: static analyzers have no runtime mechanism to opt out of these transformations, preventing tools from analyzing the raw, unfiltered control flow graph when needed.

Unconditional Pre-Analyses

The standard entry point for an analyzer within the legacy framework is the `ClEasy` interface. Before this interface hands control over to the implemented analyzer, it unconditionally executes four mandatory pre-analyses: building the call graph, finding loop-closing edges, performing a points-to analysis, and resolving variable liveness. Because there is no lazy evaluation or feature-flag mechanism, every analyzer is forced to pay the full computational cost of these operations, even for a trivial tool, such as a smoke test, that requires none of the resulting data.

IR Pollution and the God Object Anti-Pattern

The root cause of the mandatory pre-analyses is the design of the Code Storage data structures. The legacy architecture violates the Single Responsibility Principle by embedding the results of pre-analyses directly as member fields within the core Intermediate Representation (IR) nodes, including the `Insn`, `Fnc`, and `Storage` structures. For instance, a function node (`Fnc`) acts simultaneously as the IR representation of a function, a points-to analysis result container, and a call-graph membership record. This tightly couples the program representation with analysis data, creating circular ownership: a function owns a points-to graph that holds a raw pointer back to the function. Because these analysis results have no external storage mechanism, they must be eagerly computed and pre-populated into the IR before handoff.

One-Way Handoff

Which brings us to the final architectural limitation: the legacy pipeline culminates in a strict, one-way handoff. The analyzer is provided with a read-only reference to the fully populated code model. Once execution enters the analyzer, it has no structured query interface to communicate results back to the framework, nor can it dynamically request auxiliary services. For example, if an analyzer discovers a suspicious function during execution and wishes to generate a visual DOT dump of its control flow graph, it cannot do so because diagnostic outputs must be strictly pre-configured via compiler flags at invocation time.

To resolve these constraints, a modernized architecture is required. One that decouples the core IR from analysis data, evaluates annotation services on demand, and provides a flexible query interface. The design of this modernized infrastructure is the subject of the following chapter.

Chapter 3

Design

This chapter details the architectural modernization of the Code Listener infrastructure. Building upon the limitations identified in the previous chapter, the new design completely overhauls the data extraction and processing pipeline while preserving full compatibility with the existing analysis tools.

To effectively resolve the constraints of the legacy implementation, the modernized architecture was designed with five primary goals:

1. **Clean Intermediate Representation (IR):** The core Code Model must remain a pure representation of the parsed program, strictly decoupling the IR from any embedded analysis results to eliminate the "Large Class" anti-pattern [13].

2. **On-Demand Annotation Services:** The mandatory, eagerly evaluated pre-analyses must be replaced by a lazy evaluation system where auxiliary data is computed and cached only when explicitly requested by an analyzer.

3. **Unified Exporter Pipeline:** The infrastructure must provide a uniform mechanism to export the internal representation into various diagnostic and persistent formats (such as JSON, DOT, and pretty-printed text) from a single source of truth.

4. **Compiler and Analyzer Independence:** Strict boundary interfaces, later introduced as `IFrontend` and `IAnalyzer`, must be introduced to fully decouple the core infrastructure from both the underlying compiler and the target analysis tools.

5. **Backward Compatibility:** Existing analyzers, specifically the Predator tool suite, must be able to operate seamlessly on the new infrastructure without requiring any modifications to their own source code.

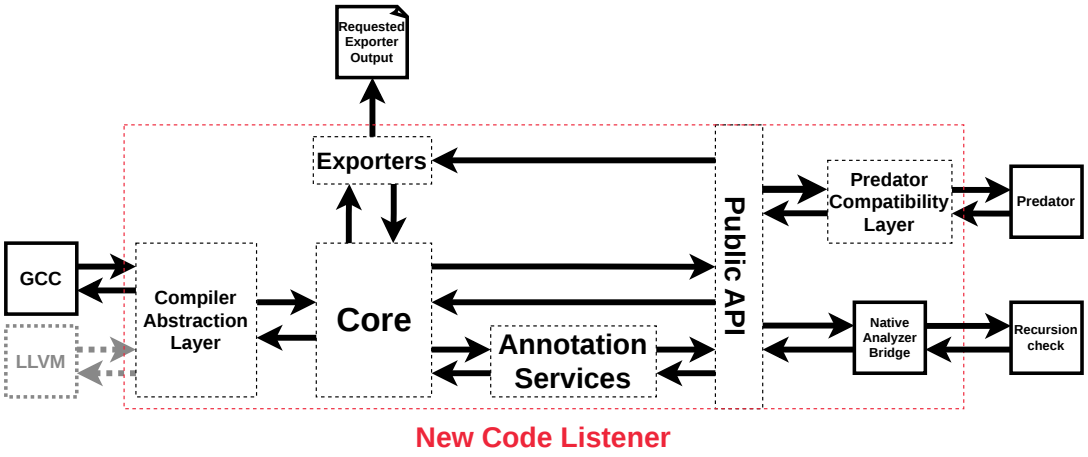


Figure 3.1: Block diagram of the New Code Listener architecture. The Compiler Abstraction Layer translates GCC GIMPLE into the compiler-independent Code Model. The Core provides the Code Model, Analysis Manager, Annotation Services, and Diagnostic Reporter. Exporters produce DOT, JSON, and pretty-printed text (PP) output. The Public API separates the engine from the built-in bridges: the Predator Compatibility Layer bridges to unmodified Predator via PredatorAdapter, while the Native Analyzer Bridge loads third-party analyzers via `dlopen`.

As illustrated in Figure 3.1 on page 14, the new architecture models the data flow sequentially from left to right. The process begins with the compiler (e.g., GCC), which passes its internal representation to the Compiler Abstraction Layer. This layer is responsible for translating the compiler-specific data into the unified, persistent Code Model housed within the Core.

It should be noted that the initial architecture was designed with a planned Model Builder abstraction layer between the GCC Adapter and the Code Model. This was intended to elegantly decouple the incremental per-function GIMPLE translation from the flat pool structure of the Code Model. However, as this proved unfeasible within the timeline of this thesis, the implemented architecture shown above omits this component, allowing the GCC Adapter to populate the Code Model directly.

Once the Code Model is populated, it serves as the central, immutable hub of the infrastructure. From here, Annotation Services can dynamically augment the model with auxiliary data, and Exporters can serialize the model to disk. Finally, execution crosses the Public API boundary, routing the model either through the Predator Compatibility Layer to service legacy tools, or through the Native Analyzer Bridge for modern, dynamically loaded analyzers.

The following sections break down each of these architectural components in detail, mirroring the flow of data through the system.

3.1 Compiler Abstraction Layer

The Compiler Abstraction Layer is the first stage of the processing pipeline. Its primary responsibility is to intercept the compilation process, extract the compiler's internal representation, and translate it into a unified, compiler-independent format. Crucially, this is

the only component in the entire architecture that interacts with the underlying compiler API.

The GCC Plugin Lifecycle

The infrastructure integrates with GCC via its standard plugin API. The lifecycle begins at `plugin_init()`, which is called when the plugin is dynamically loaded by GCC and where compiler version checks are performed. A custom GIMPLE pass is then registered to execute immediately after the compiler's native `cfg` (Control Flow Graph) pass. This is the same insertion point used by the legacy Code Listener.

However, the execution flow diverges significantly from the legacy architecture. Instead of analyzing the code function-by-function during the `cfg` pass, the new design merely captures the incoming GIMPLE data. The actual translation and subsequent analysis are deferred until the `PLUGIN_FINISH` event, which fires only after the entire translation unit has been parsed and before GCC finalizes its internal state and finally exits [4].

Isolation and the GCCAdapter

To achieve strict modularity, all GCC-specific logic is encapsulated within a dedicated compiler abstraction layer. When the `PLUGIN_FINISH` event is triggered, the `GCCAdapter` module performs a one-time, comprehensive translation of the GCC GIMPLE trees into the central Code Model. Once this translation is complete, the Code Model contains no compiler-specific data. This strict boundary ensures that no GCC-specific headers or data structures leak into the rest of the system.

Furthermore, even diagnostic reporting is abstracted. A dedicated `GCCDiagnosticReporter` class wraps GCC's native error-reporting functions. This ensures that any defects found by downstream analyzers are emitted with the correct file, line, and column information in the standard compiler error format, without the analyzers themselves needing to know they are running inside GCC.

Modernized Plugin Arguments

The legacy configuration structures were replaced by a new, cleanly separated argument parsing architecture. In the legacy implementation, listeners were unconditionally forced to use specific filters through a hardwired pipeline configuration. The new design maps user arguments cleanly to optional features without hidden side effects, allowing command-line flags to specifically control the exporter and analysis pipeline on demand.

The `IFrontend` Strategy Pattern

To fully decouple the analysis backend from the GCC compiler, the extraction process is generalized using the Strategy design pattern [3] via the `IFrontend` interface. The `IFrontend` exposes a single virtual contract:

```
virtual bool run(std::vector<std::unique_ptr<IAnalyzer>>&, AnalysisContext&) = 0;
```

There are two concrete implementations of this interface:

1. `GCCFrontend`: Operates as the live GCC plugin, invoking the GCCAdapter to generate the model directly from memory.

2. `JSONFrontend`: Operates as a standalone tool, reading a previously exported JSON representation of the model from disk.

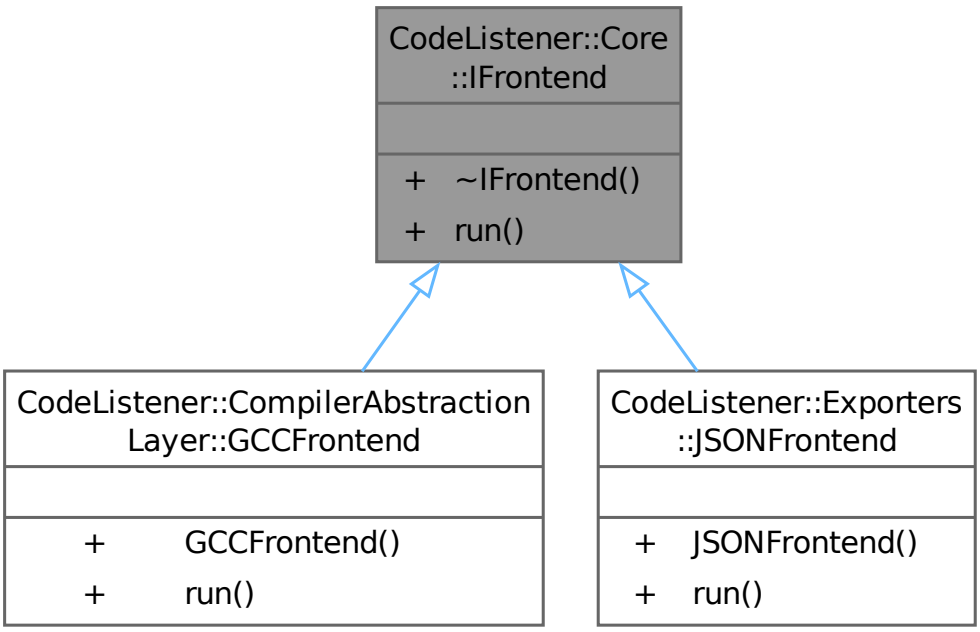


Figure 3.2: Class diagram of the `IFrontend` Strategy pattern [3]. The interface abstracts the source of the program model, allowing the analysis backend to operate identically whether running as a live GCC compiler plugin via `GCCFrontend` or as a standalone offline tool parsing JSON data via `JSONFrontend`.

Once the active frontend finishes populating the Code Model, it iterates through all registered analyzers and invokes the analysis backend. Because both frontends implement the same contract, the live compiler plugin and the offline standalone tool are completely interchangeable from the perspective of the analysis tools.

3.2 Core — Code Model and Query System

At the heart of the modernized infrastructure is the `CodeModel`. Once the Compiler Abstraction Layer intercepts and translates the compiler's intermediate representation, the `CodeModel` serves as the central, immutable, and parser-independent representation of the program.

Resolving IR Pollution and the Large Class Anti-Pattern

The most critical architectural shift in the `CodeModel` is its strict adherence to the Single Responsibility Principle. As identified in the legacy architecture Section 2.3 on page 9, the original data structures, namely the `Fnc`, `Insn`, and `Storage`, suffered from severe Intermediate Representation (IR) pollution, acting as "Large Class" [13] that directly embedded the results of mandatory pre-analyses such as points-to and call graphs, and also variable liveness sets.

The new `CodeModel` contains absolutely zero analysis data. It is a pure, structural representation of the parsed source code. By stripping out all embedded analysis fields,

such as `ptg` or `callGraph`, the circular ownership and memory management complexities of the legacy design are eliminated. Any auxiliary analysis results are now computed and managed entirely externally by the Annotation Services.

Entity Pools and ID-Based References

To efficiently store the program's data, the `CodeModel` employs an Entity Pool pattern inspired by Data-Oriented Design (DOD) principles. Instead of allocating individual nodes on the heap and linking them with fragile raw pointers, the model is flattened into five central pools: types, variables, functions, basic blocks, and instructions. These pools use `std::deque<T>` containers, which guarantee that memory addresses remain stable as the pools grow dynamically.

Entities reference each other exclusively through lightweight, strongly typed identifiers: `TypeId`, `VariableId`, `FunctionId`, `BlockId`, and `InstructionId`. This ID-based referencing provides several major advantages: it completely eliminates the risk of dangling pointers, vastly simplifies memory ownership, and trivially enables the serialization of the entire model, such as exporting the parsed program to JSON.

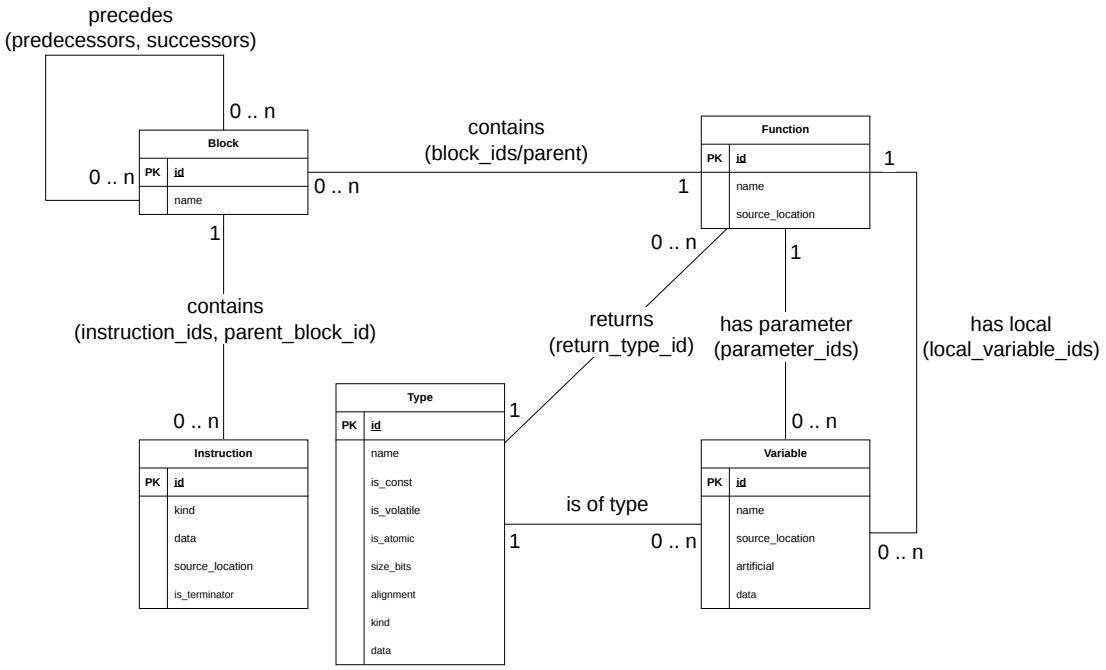


Figure 3.3: Entity relationship diagram of the Code Model. The model uses a flat, data-oriented structure in which all types, variables, functions, blocks, and instructions reside in centralized pools. Entities navigate the graph by storing lightweight IDs. For example, a `Function` holds a vector of `BlockId`s rather than raw memory pointers, ensuring safe serialization and decoupling.

Modernized Type and Instruction Hierarchies

The intermediate representation models the exhaustive, closed set of types and instructions expressible in the C programming language. To achieve safe and efficient polymorphism

without the overhead of deep class inheritance and dynamic virtual dispatch, the new design utilizes `std::variant`-based wrappers.

- **Type Hierarchy:** The `Type` entity acts as a wrapper over `TypeData`, which holds a `std::variant` of concrete types such as `PointerType`, `StructType`, `ArrayType`, or `VoidType`.
- **Instruction Hierarchy:** Similarly, the `Instruction` entity wraps `InstructionData`, a `std::variant` over operations like `AssignInstruction`, `CallInstruction`, and `SwitchInstruction`.

Notably, because the new architecture does not force hardwired filters upon the analyzer, `SwitchInstruction` is preserved natively within the `CodeModel`. Analyzers that require raw switch semantics can now access them directly, whereas previously they were unconditionally forced to consume chains of conditional jumps.

The Query System

Rather than introducing a separate, disjoint "Query System" component, the querying mechanisms are seamlessly built directly into the public interface of the `CodeModel`. The API provides direct $O(1)$ lookups by ID. These include, but are not limited to, functions such as `getType(TypeId)` and `getFunction(FunctionId)`, alongside modern C++20 range-based traversal views.

This allows an analyzer to intuitively and efficiently traverse the control flow graph using standard range-based `for` loops. For example, iterating over all instructions within all blocks of a function can be concisely expressed using `model.blocksOf(func)` and `model.instructionsOf(block)`. This exposes the complex underlying data structures through a clean, highly readable interface.

3.3 Annotation Services

In the legacy Code Listener infrastructure, the `ClEasy` interface forced the unconditional execution of four mandatory pre-analyses, specifically the call graph construction, loop-closing edge detection, points-to analysis, and variable liveness, before handing control over to the analyzer. This eager evaluation wasted computational resources and forced the core Intermediate Representation (IR) to act as a container for analysis results.

The modernized architecture resolves this by introducing a lazy evaluation system driven by Annotation Services. In this design, the `CodeModel` remains a pure structural representation of the program, while auxiliary analysis results are computed exclusively on demand and cached externally.

The `AnalysisManager` and Lazy Evaluation

At the core of this system is the `AnalysisManager`. Owned by the `AnalysisContext`, this manager is passed to every analyzer and exporter in the pipeline. It acts as a type-erased cache, internally utilizing an `unordered_map` to map unique analysis keys to instantiated annotation objects.

When an analyzer requires a specific set of data, it queries the manager using a template method, typically formatted as:

```
template <typename T>
const T& getAnnotation(const CodeModel& model);
```

When this method is invoked, the `AnalysisManager` checks its internal cache. If the requested annotation type `T` has already been computed, it immediately returns the cached reference in $O(1)$ time. If it has not yet been computed, the manager invokes the static factory method `T::build(model)` to generate the data, stores it in the cache, and then returns the reference. Because the `AnalysisManager` is shared via the `AnalysisContext`, an expensive annotation, such as a call graph, computed by an exporter can be seamlessly reused by a subsequent static analyzer without being recomputed.

Type-Safe Registration via CRTP

To ensure type safety and eliminate the need for fragile string-based lookups or dynamic casting, the architecture utilizes the Curiously Recurring Template Pattern (CRTP). Every annotation service must inherit from a templated base class, `AnnotationBase<Derived>`, passing its own type as the template parameter.

This CRTP base automatically injects two critical components into every service:

1. A strictly enforced signature for the `build(model)` static factory method.
2. A static `AnalysisKey` member. The memory address of this static member (`&Derived::Key`) serves as the unique identifier for the type-erased cache. This elegant, low-overhead pattern guarantees globally unique keys per annotation type and heavily mirrors the `AnalysisInfoMixin` design found in modern LLVM [1].

[1] https://llvm.org/doxygen/structllvm_1_1AnalysisInfoMixin.html#details

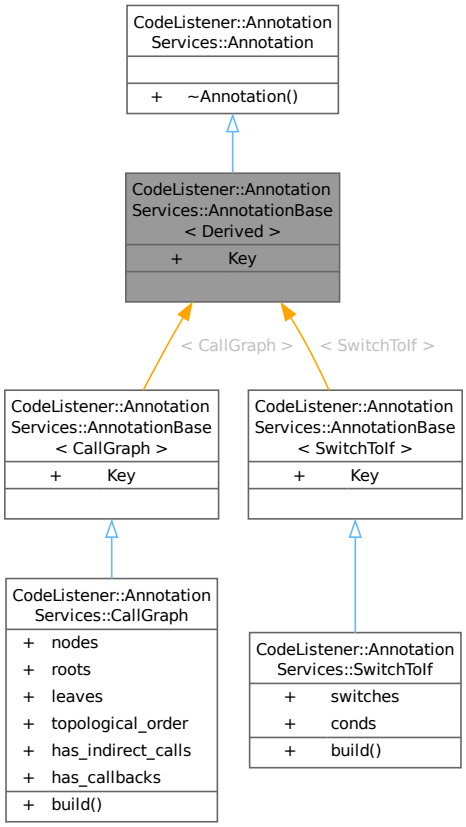


Figure 3.4: Class diagram illustrating the Curiously Recurring Template Pattern (CRTP) used for Annotation Services. Inheriting from `AnnotationBase` guarantees that each service exposes a unique `AnalysisKey` and a standard `build()` factory, allowing the `AnalysisManager` to instantiate and cache them generically.

Implemented Services

Currently, the infrastructure implements two primary annotation services:

- **CallGraph:** This service analyzes the `CodeModel` to construct a directed graph of function invocations. Using Tarjan's strongly connected components (SCC) algorithm [14], it computes topological orderings, identifies root functions, which are functions with no callers, and leaf functions.
- **SwitchToIf:** This service computes synthetic variable names and block-label allocations required to safely unfold `SWITCH` instructions into chains of conditional jumps. Crucially, unlike the legacy `unfold_switch` filter, which destructively mutated the IR, this service only precomputes the textual rendering data. The pure `SwitchInstruction` remains perfectly intact within the `CodeModel` for any analyzer that natively supports it.

It is worth noting that the legacy pre-analyses for variable liveness, points-to graphs, and loop-closing edges were not ported as individual annotation services. Because the modernized infrastructure preserves the purity of the `CodeModel`, advanced analyzers, such as the Predator verification kernel, prefer to execute their own highly specialized, internal versions of these analyses rather than rely on generic framework approximations.

3.4 Exporters and Standalone Tools

To satisfy the requirement for robust diagnostic outputs and persistent storage, the modernized architecture introduces a unified exporter pipeline. Rather than generating outputs through disjoint, parallel listeners as the legacy system did, the new design serializes the central `CodeModel` into multiple formats, those being the DOT, JSON, and pretty printed text (PP), using a standardized, object-oriented exporter hierarchy.

The Exporter Base Class and Design Patterns

All export functionality is rooted in the abstract `Exporter` base class. This class orchestrates the export process by combining three classical design patterns:

- **Visitor Pattern:** The `Exporter` inherits from a `CodeModelVisitor` interface, allowing it to systematically traverse the hierarchical structure of the `CodeModel`, those being the functions, blocks, instructions, and operands, without modifying the entity classes themselves.
- **Template Method Pattern:** The base class defines the overarching traversal algorithm, providing virtual hooks that the concrete subclasses override to emit format-specific output.
- **Strategy Pattern:** The active exporter is selected dynamically at runtime via the `AnalysisContext`. The export methods, specifically either `exportDot()`, `exportJson()`, or `exportPP()`, allow the infrastructure to swap export strategies seamlessly.

Furthermore, while the main analysis pipeline shares a single `AnalysisManager` instance, each concrete `Exporter` is instantiated with its own independent `AnalysisManager`. This allows the exporter to lazily request and utilize auxiliary data, such as the `SwitchToIf` annotation, internally without polluting the primary analysis cache.

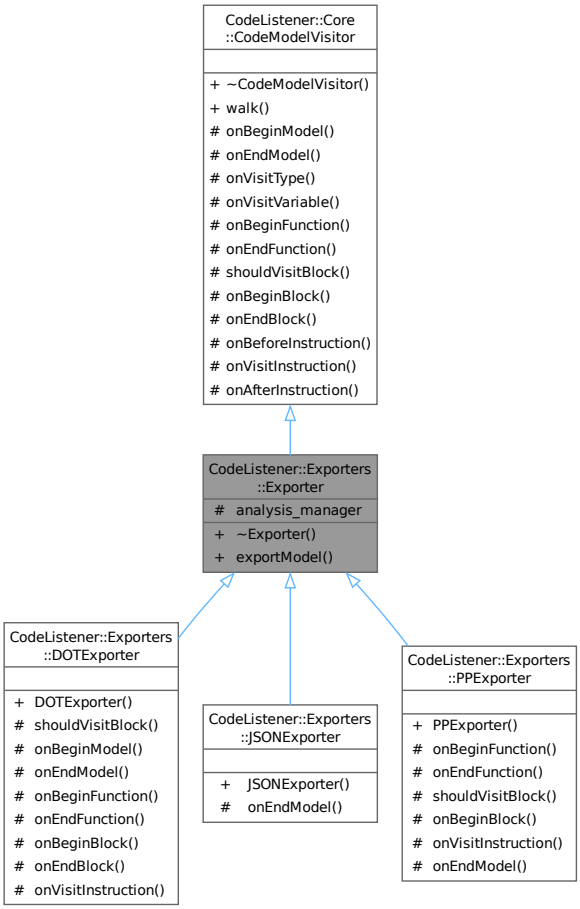
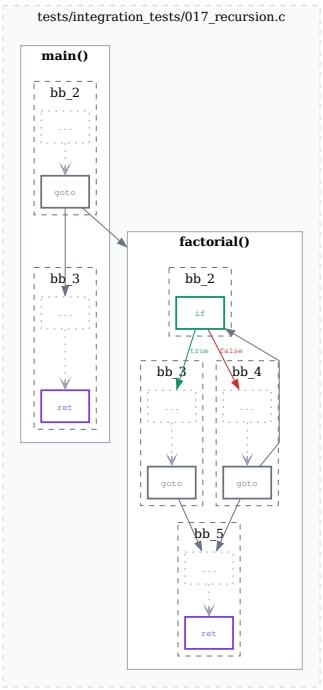


Figure 3.5: Class diagram of the Exporter hierarchy. The Exporter utilizes the Visitor and Template Method patterns to traverse the Code Model. Concrete implementations, specifically `DOTExporter`, `JSONExporter`, or `PPExporter`, provide specific serialization formats.

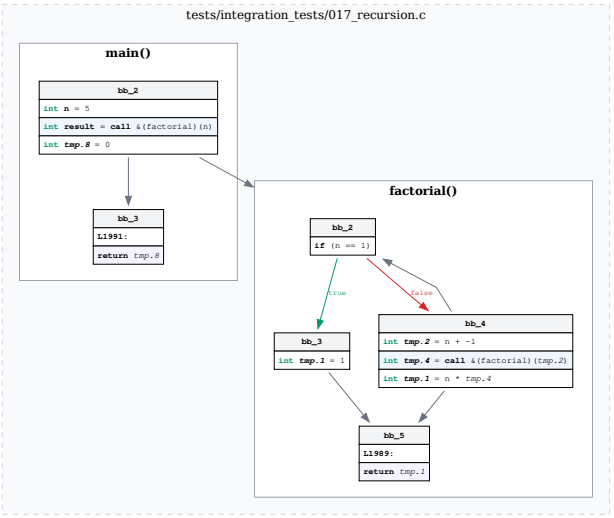
Diagnostic and Visual Exporters

The infrastructure implements two exporters primarily designed for human inspection and diagnostics:

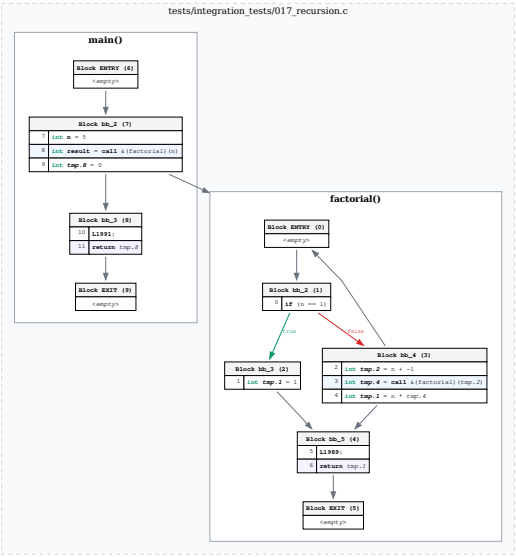
- **DOTExporter:** This component generates a Graphviz `.dot` representation of the Control Flow Graph (CFG) for each parsed function. It is parameterized by a `DotVerbosity` flag, that allows the user to toggle the inclusion of detailed type information and unique variable identifiers. This fulfills the program graph visualization requirement, providing analysts with immediate visual confirmation of the extracted execution paths [11].
- **PPExporter:** Serving as a direct, modernized replacement for the legacy `cl_pp` listener, the Pretty Printer exporter outputs a human-readable, three-address code (3AC) listing of the program.



(a) `CLEAN` has block structure only, instruction types are shown as keywords, and mirrors the legacy look.



(b) `COMPACT` has full instruction text and operand names, ENTRY/EXIT pseudo-blocks are suppressed.



(c) `FULL` adds block IDs, source line numbers, and ENTRY/EXIT pseudo-blocks.

Figure 3.6: Control Flow Graphs of a recursive `factorial` program generated by the `DOTExporter` at the three available verbosity levels. `CLEAN` is suited for quick topology inspection, `COMPACT` for human-readable analysis, and `FULL` for debugging the extractor itself.

Persistent Storage and the JSON Round-Trip

A major limitation of the legacy architecture was its strict confinement to the live compiler memory space. To enable offline processing and cross-translation-unit (whole-program) analysis, the new infrastructure introduces the `JSONExporter` and `JSONImporter`.

Utilizing the `nlohmann/json` library [2], the `JSONExporter` serializes the entire CodeModel, including all entity pools and ID-based cross-references, into a persistent JSON file. The `JSONImporter` performs the inverse operation, safely deserializing the JSON file back into a fully functional `CodeModel` in memory.

Because the C and C++ compilation models operate on independent translation units (source files), full verification often requires analyzing the entire program at once. To facilitate this, the `ModelMerger` component was introduced. It accepts an arbitrary number of deserialized `CodeModel` objects and safely merges their distinct entity pools into a single, unified whole-program `CodeModel`.

Standalone Tools

By leveraging the JSON round-trip and the `IFrontend` Strategy pattern, the infrastructure provides two standalone, offline command-line tools:

1. **cl_merge:** A utility that parses multiple `.json` files exported during a build process, invokes the `ModelMerger`, and outputs a single, consolidated JSON representation of the entire software project.

2. **cl_analyze:** A standalone analysis driver. Instead of running as a live GCC plugin, it utilizes the `JSONFrontend` to load a pre-merged `CodeModel` from disk. It then invokes the registered analysis backend identically to the `GCCFrontend`. From the perspective of the static analyzers, there is absolutely no difference between running live inside GCC or running offline via `cl_analyze`.

3.5 Public API and Predator Compatibility Layer

The final stage of the modernized architecture is the handoff to the actual static analysis tools. To isolate the core infrastructure from the specific requirements of any single analyzer, the system exposes a strict public boundary via the `IAnalyzer` interface.

The `IAnalyzer` interface dictates a single, simple contract:

```
virtual void analyze(const CodeModel&, AnalysisContext&) = 0;
```

The framework provides two built-in implementations of this interface that bridge the gap between the internal Code Model and the external analysis tools: the `NativeAnalyzerBridge` for modern plugins, and the `LegacyPredatorBridge` for backward compatibility.

[2] https://github.com/nlohmann/json

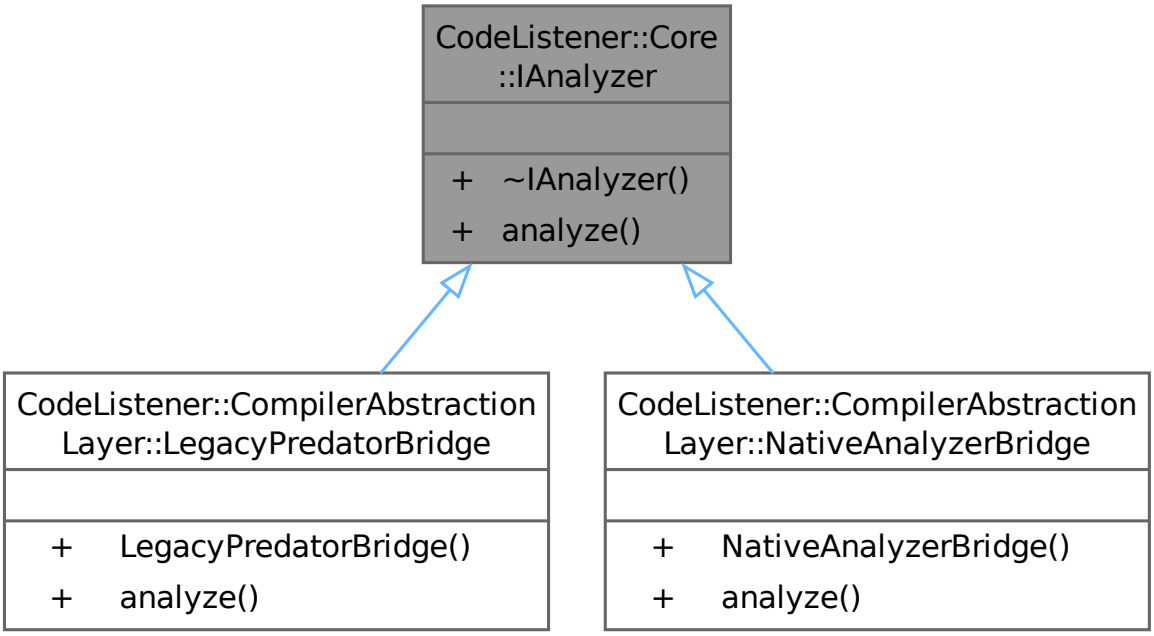


Figure 3.7: Class diagram of the `IAnalyzer` interface. The `NativeAnalyzerBridge` provides a modern integration path for dynamically loaded tools, while the `LegacyPredatorBridge` provides a backward-compatibility layer.

Native Analyzer Bridge

The `NativeAnalyzerBridge` provides the integration path for new, third-party static analyzers. An external analyzer is compiled as a shared library (`.so`) that exports a standardized C-ABI structure called `cl_native_analyzer_api_t`. The GCC plugin loads this library at runtime using `dlopen`, which is configured via the `-load-analyzer` command-line flag.

Crucially, when the `NativeAnalyzerBridge` delegates execution to the dynamically loaded analyzer, it forwards the complete `AnalysisContext`. This means the external analyzer has full, unrestricted access to the `AnalysisManager`; by extension, it can lazily request annotations like the `CallGraph`, use the `DiagnosticReporter` to emit compiler-formatted warnings, and also use the unified export helpers. To demonstrate this modern capability, the infrastructure includes a reference `RecursionCheck` analyzer, that successfully uses the `CallGraph` annotation to detect cyclic function calls on demand.

Legacy Predator Compatibility and the Adapter Pattern

The most critical constraint of the modernization effort was ensuring that the VeriFIT group's primary tool, the Predator shape analyzer, continued to function without requiring any modifications to its own source code. Predator expects its input in the form of the legacy `cl_code_listener` C API, which is a stream of sequential function-pointer callbacks such as `fnc_open`, `insn`, or `fnc_close`.

To bridge this fundamental architectural gap, the infrastructure implements the classic Adapter design pattern [3]. The `LegacyPredatorBridge` instantiates a `PredatorAdapter`, which systematically traverses the new, object-oriented `CodeModel` and translates its entities back into the legacy stream of C callbacks.

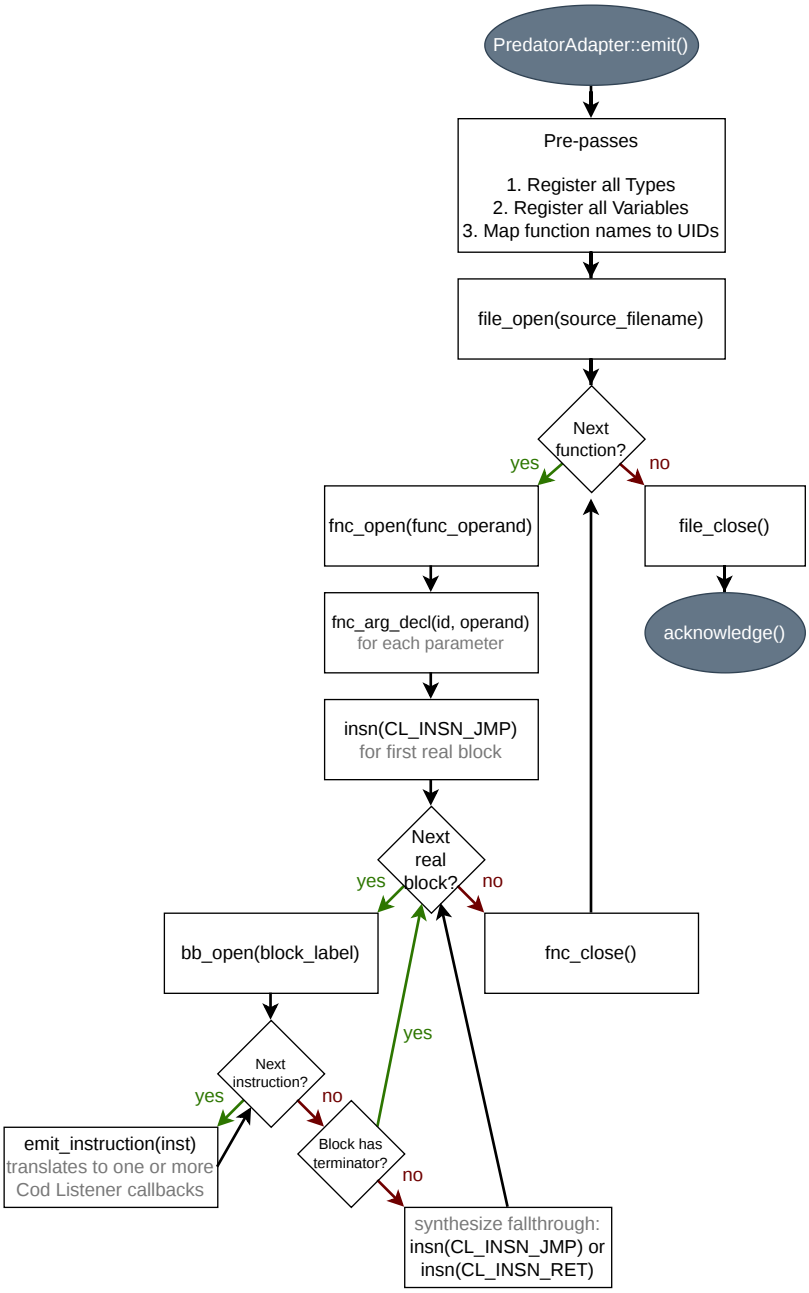


Figure 3.8: Data flow diagram of the Predator Compatibility Layer. The `PredatorAdapter` structurally adapts the modern `CodeModel` to fulfill the legacy `cl_code_listener` callback interface expected by Predator, seamlessly preserving backward compatibility.

Capability Asymmetry and the Illusion of Pre-Analyses

An important architectural asymmetry exists between the two bridges. While the `NativeAnalyzerBridge` exposes the full `AnalysisContext` to modern tools, the `LegacyPredatorBridge::analyze()` method explicitly discards the context. Because Predator predates the `AnalysisManager`, it cannot dynamically request framework annotations.

Instead, the `PredatorAdapter` relies solely on the `CallGraph` annotation service to determine the topological ordering of root functions (functions with no callers), ensuring

they are emitted to Predator in the correct sequence. Once the adapter fires the final `acknowledge()` callback, the handoff is complete.

This elegantly resolves the "mandatory pre-analyses" problem identified in the legacy architecture. The modernized framework no longer computes points-to sets, loop-closing edges, or variable liveness sets. Predator's regression tests continue to pass seamlessly because the Predator kernel has always been capable of computing its own specialized versions of these analyses internally once the callback stream concludes. The legacy framework's eager computation of these datasets was, ultimately, redundant for advanced tools.

Chapter 4

Implementation

Building on the conceptual and architectural design outlined previously, this chapter details the practical implementation of the modernized Code Listener. It emphasizes the engineering decisions, programming techniques, and build systems that translated the object-oriented design into a functional prototype.

The structure of this chapter follows the software's logical execution path. Section 4.1 addresses the development environment and build artifacts, while Section 4.2 describes the physical source code organization. Subsequent sections cover the live GCC plugin pipeline (Section 4.3), the compiler-independent core and analyzer interfaces (Section 4.4), and the offline serialization path (Section 4.5). The chapter concludes with implemented testing and tooling support (Section 4.6), current extension points and limitations (Section 4.7), and the size of the authored implementation (Section 4.8). In accordance with software engineering best practices, exhaustive source code listings are omitted from the main text and are instead provided in the GitHub repository [1].

4.1 Development Environment and Platform Constraints

Developing a compiler plugin imposes strict platform and toolchain constraints. Like the original Predator toolchain, the modernized infrastructure targets Linux environments. Because the plugin is compiled as a shared object and loaded directly into the GCC process memory space, it is intrinsically tied to the GCC Plugin API. The build system must match the exact GCC installation used at runtime and compile against that specific version's plugin headers. This constraint is managed via a `TARGET_GCC` CMake variable. By default, it is set to `gcc-12`. If the required compiler headers are missing during configuration, the build is designed to abort immediately rather than producing a partially or silently broken binary.

The implementation utilizes C++23, the latest stable version of the C++ standard available during development [5]. This choice enables the internal models to leverage standard library facilities, thereby avoiding reliance on complex, custom framework code. For instance, `std::variant` was employed for closed type and instruction hierarchies, `std::optional` was used to safely handle incomplete compiler metadata, and modern range-based traversal was applied where appropriate within the Code Model.

To ensure safety and compatibility, the generated binaries are compiled conservatively. All plugin-facing targets are compiled with `-fPIC` to support dynamic loading. Further-

[1] https://github.com/pseja/gcc-plugin-for-static-analyzer-support

more, the codebase uses `-fno-rtti` to avoid the overhead of runtime type information, and enforces strict compilation checks via `-Wall`, `-Wextra`, `-Werror`, and `-pedantic` to catch potential defects at compile time.

External dependencies were kept to an absolute minimum. Persistent JSON storage is handled by the header-only `nlohmann/json` library, avoiding the introduction of shared-library dependencies that could cause symbol collisions inside the GCC process [10]. Additionally, the standard `patch` utility is required during the build configuration to automatically apply compatibility shims to the legacy Predator codebase, ensuring the experiment is reproducible from a clean repository checkout.

4.2 Build System and Source Code Organization

A two-layered build system is employed in the implementation. CMake manages the complex target graph, dependencies, and test registrations, while a top-level Makefile offers concise commands for standard development workflows, including `make build`, `make test`, `make json`, `make callgraph`, and others.

The build process produces five distinct categories of artifacts:

1. `cl_core`: The compiler-independent static library housing the Code Model.
2. `cl_gcc`: The shared library that GCC dynamically loads as the actual plugin.
3. **Analyzer Libraries:** Dedicated shared libraries including `cl_callgraph_dot` and `cl_recursion_check`, with additional analyzers built in the same way.
4. **Standalone Executables:** The offline tools `cl_merge` and `cl_analyze`.
5. **Auxiliary Outputs:** Generated Doxygen documentation, class diagrams, and CTest registrations.

To maintain the architectural boundaries defined in the design phase, the physical directory layout strictly mirrors the separation of concerns:

- `src/core/`: Contains the compiler-independent Code Model, entity pools, and analyzer interfaces.
- `src/adapters/gcc/`: Houses the GCC plugin registration, the GIMPLE translation logic, and argument parsing.
- `src/adapters/predator/`: Contains the legacy compatibility layer for Predator.
- `src/annotation_services/`: Implements the lazily evaluated analyses, most importantly the Call Graph service used by multiple analyzers.
- `src/exporters/`: Contains the DOT, JSON, and pretty-printed export pipelines, together with the JSON frontend and merger.
- `src/tools/`: Holds the entry points for the `cl_merge` and `cl_analyze` executables.
- `include/`: Exposes the C and C++ analyzer Application Binary Interfaces (ABIs).
- `tests/`: Contains the native regression test suites.

This directory structure enforces the most critical implementation rule of the entire project: GCC headers must never leak into compiler-independent code. All direct dependencies on the GCC API are quarantined entirely within `src/adapters/gcc/`. This physical boundary makes it possible to compile the offline `cl_analyze` tool and the Code Model without requiring the GCC runtime data structures.

4.3 Realizing the GCC Plugin Pipeline

The live compiler plugin represents the first of the three primary execution paths. Its entry point was implemented in `register_plugin.cpp`. Its sole responsibility is to verify that the runtime GCC version matches the compiled version before delegating initialization to the `PluginContext` singleton. Inside `PluginContext.cpp`, the plugin registers its custom GIMPLE pass immediately following GCC's native `cfg` pass. It records the current input file at the `PLUGIN_START_UNIT` event and finally installs a `PLUGIN_FINISH` callback that triggers the model export and analyzer invocation at the end of the compilation unit.

Command-line argument parsing is centralized in `PluginArgs.cpp`. It parses the raw GCC plugin arguments, converts typed values, like exporter verbosities, and validates them. The arguments currently wired into runtime behavior cover verbosity selection, analyzer loading, forwarded analyzer arguments, and the JSON, DOT, and PP export requests. The parser also recognizes flags intended as future work hooks, such as `help`, `version`, `dry-run`, `dump-types`, `preserve-ec`, `pid-file`, and `type-dot`, preventing the need to hardwire ad hoc conditionals throughout the plugin logic later.

The custom GCC pass is defined in `Pass.cpp`. For every function, its execute method forwards work directly to `GCCAdapter::processFunction`. `GCCAdapter.cpp` is the largest single implementation file in the project. It incrementally populates the new Code Model by traversing the GCC trees and GIMPLE statements. To optimize this translation:

- Compiler types are memoized using a cache keyed by the GCC tree nodes, ensuring that recursive types are represented only once.
- Anonymous compiler aggregates are assigned synthetic location-based names, significantly improving the readability of diagnostic and JSON outputs.
- Source locations are aggressively normalized via helper functions to recover coordinates even for GIMPLE statements missing complete metadata, providing the best possible information for diagnostics and analysis.
- The adapter stores only internal identifiers, entirely stripping away the raw GCC memory pointers so that the resulting Code Model outlives the compiler process.

Finally, diagnostic messages are routed through `GCCDiagnosticReporter.cpp`. This class maps framework diagnostics into native GCC calls, specifically `inform`, `warning_at`, `error_at`, `fatal_error`, ensuring that any bugs found by an analyzer appear to the user exactly like standard compiler errors. Since GCC has no built-in debug-level reporting channel, the plugin provides a development-only path that prints grey `stderr` lines containing the relative file path, line number, function name, and message text. These messages are emitted only when the verbosity level is set to 2 or higher, allowing adapter debugging to remain available on demand without affecting ordinary analyzer runs.

4.4 Core Representation and Analyzer Interfaces

The internal Code Model is driven by centralized entity pools. In the implementation, these pools for types, variables, functions, blocks, and instructions are instantiated as `std::deque` containers. A `std::deque` allocates memory in fixed chunks, meaning that as the GCC adapter incrementally populates the model and holds temporary references, the memory addresses of previously inserted entities remain perfectly stable [1]. This avoids the pointer-invalidation issues common to `std::vector` reallocations.

Because deep inheritance trees were avoided, instruction and type payloads are implemented using C++23 `std::variant`. This allows each instruction to be stored as a plain value type. Later processing, such as exporting or analyzing the model, is performed cleanly by branching over the closed set of alternatives using `std::visit`.

To load the analysis logic, `PluginContext::load_analyzer` performs dynamic library loading. It explicitly checks the library for the modern `cl_get_native_api` symbol. If the symbol exists and versions match, the library is wrapped in the `NativeAnalyzerBridge`. If the modern API is missing, the infrastructure falls back to the historical C ABI through the legacy `cl_get_analyzer_api` entry point. The resulting listener is then wrapped in a `LegacyPredatorBridge`. Missing symbols or version mismatches are reported as errors via the plugin diagnostic channel and prevent the analyzer from loading.

Modern native analyzers are provided with an `AnalysisContext` declared in the public analyzer headers. Its implementation is located in `src/exporters/AnalysisContext.cpp`. This context provides three specific services: the diagnostic reporter, the `AnalysisManager`, and unified export helpers exposed through `exportDot`, `exportPP`, and `exportJson`. Because the `AnalysisManager` uses lazy evaluation, multiple analyzers that run sequentially can request and share expensive derived data, such as the Call Graph, from a single cached instance.

Conversely, backward compatibility is handled by `PredatorAdapter.cpp`. This adapter systematically walks the modern Code Model and reconstructs the legacy `cl_type` and `cl_var` structures. The emission sequence was carefully implemented to match historical expectations: all types and variables are pre-registered before any function bodies are streamed. This exact ordering ensures the legacy Predator verification kernel functions entirely unmodified.

4.5 Serialization and Offline Whole-Program Workflow

The second major execution path is the offline workflow, which completely decouples the analysis from the live compiler process. Serialization is implemented via the `JSONExporter` and `JSONImporter` classes. Because the model relies solely on strongly typed IDs rather than memory addresses, the serialization process maps internal IDs to JSON values almost directly, avoiding complex graph-pointer reconstruction.

Whole-program analysis requires merging multiple translation units into a single model. This is executed by `ModelMerger.cpp`. To prevent ID collisions, for example, `FunctionId(0)` existing in two separate files, the merger generates explicit remapping tables. As a new file is appended to the consolidated Code Model, the merger systematically rewrites every cross-reference through these tables, seamlessly preserving the integrity of the graph topology across file boundaries.

These capabilities are exposed to the user via two standalone executables. The `cl_merge` tool accepts multiple `.json` files, leverages the `ModelMerger`, and outputs a unified pro-

gram model. The `cl_analyze` tool acts as the offline analysis driver. It can either load a native analyzer library or invoke the built-in offline exporters for DOT and pretty-printed output. Instead of using the `GCCDiagnosticReporter`, it instantiates an equivalent `StderrDiagnosticReporter`. When the analyzer path is selected, it loads the target analyzer and feeds it the deserialized JSON model. Consequently, the analyzer backend requires no modifications to operate offline; the semantic contract remains identical whether the model originated from GCC memory or a JSON text file.

4.6 Testing, Tooling, and Practical Workflow

To ensure the architecture serves as a robust research prototype, extensive automated testing and tooling were implemented. This section describes the implemented testing infrastructure and practical usage flow, while the quantitative evaluation itself is deferred to the following testing chapter. When the `WITH_PREDATOR` CMake option is enabled, the build system compiles the patched Predator components and exposes their regression suite through CTest.

To verify correctness across the different architectural boundaries, CTest is configured with four separate regression suites:

1. **GCC-Adapter Tests**: 46 tests validating that the GIMPLE translation produces the correct JSON output against historical fixtures.

2. **Code Model Integration Tests**: 38 tests focusing specifically on types, initializers, control flow edges, and edge cases.

3. **Offline Pipeline Tests**: 3 end-to-end tests covering the offline analysis path through the `cl_analyze` tool.

4. **Predator Regression Suite**: 1,192 tests executed against the patched Predator kernel to validate backward compatibility on the preserved listener interface.

To demonstrate how the system is used in practice, two reference native analyzers were implemented: `callgraph_dot` and `recursion_check`. The `callgraph_dot` analyzer requests the shared `CallGraph` annotation and emits its nodes and edges as a Graphviz DOT file. The `recursion_check` analyzer requests the same annotation, iterates over its strongly connected components, and reports direct and mutual recursion both to diagnostics and to a text report. A developer can run `make callgraph FILE=example.c` to generate a `callgraph.dot` visualization of the program's interprocedural structure. Following that, `make recursion FILE=example.c` checks the same program for direct and mutual recursion, and writes `recursion_report.txt` by default. In both cases, the output path can be redirected via the forwarded-analyzer argument string exposed by the top-level Makefile. Together, these analyzers show that both simple visual tools and more analysis-oriented native analyzers can be written over the new architecture with minimal boilerplate.

At this stage, the implementation already delivers three concrete execution paths: the live GCC plugin path, the Predator compatibility path, and the offline JSON-based path. The subsequent testing chapter quantitatively evaluates these execution paths, whereas the present chapter remains focused on their construction.

4.7 Current Limitations and Extension Points

While the architecture is stable, several limitations and extension points remain. The `AnalysisContext` and `AnalysisManager` are explicitly designed as extension points for future annotation services. However, the offline `cl_analyze` workflow currently only supports native analyzers or built-in exporters. Executing legacy analyzers like Predator offline is not supported because they depend heavily on the `PredatorAdapter` reconstructing the legacy data stream, a process currently tied directly to the live GCC execution path.

These limitations do not diminish the prototype's practical value; rather, they delineate the next logical steps in its development. These include expanding the set of shared annotations, implementing concrete behavior for currently reserved plugin arguments, and extending the offline path where feasible without compromising the existing analyzer boundary.

4.8 Implementation Size

To quantify the scale of the implementation and distinguish original thesis work from legacy code, the authored C and C++ implementation files comprise approximately 11,309 lines of code (LOC) [2]. The repository includes an additional 2,061 lines of automated test code, bringing the total authored contribution to 13,370 lines.

Subsystem	LOC
Core model (`src/core/`)	2,082
GCC integration (`src/adapters/gcc/`)	2,759
Predator compatibility (`src/adapters/predator/`)	1,239
Annotation services (`src/annotation_services/`)	599
Exporters and offline frontend (`src/exporters/`)	3,825
Command-line tools (`src/tools/`)	368
Native analyzers (`analyzers/`)	279
Public headers (`include/`)	158
Total implementation code	11,309

Table 4.1: Lines of code (LOC) for each major subsystem of the implementation.

In contrast, the imported Predator submodule comprises approximately 673,000 lines of legacy C and C++ code, with Code Listener itself totaling about 25,000 lines, including headers, support files, and tests. This legacy code is treated solely as an external dependency and evaluation baseline, and is excluded from the thesis implementation metrics. As illustrated in Table 4.1 on page 33, the code-size distribution reflects the architectural priorities of the project, with the largest components allocated to the Exporter pipelines, the GCC translation adapter, and the Core representation.

[2] Measured in April 2026

Chapter 5

Testing and Evaluation

This chapter evaluates the modernized Code Listener infrastructure. The primary objective is to demonstrate that the new architecture meets its design requirements: accurate extraction of program semantics, support for offline serialization, and preservation of backward compatibility at the historical Predator listener boundary.

The evaluation is divided into three parts. First, the native test suites verify the structural correctness of the new `CodeModel` and the serialization pipeline. Second, a large-scale regression analysis is performed using the legacy Predator tool to prove backward compatibility. Finally, the performance characteristics of the new architecture are critically analyzed, highlighting the trade-offs between structural overhead and algorithmic efficiency.

5.1 Testing Methodology and Environment

The testing methodology was designed to isolate the architecture's distinct boundaries. Because the infrastructure acts as a mediator between the compiler and the analyzers, defects could theoretically originate in the GCC extraction phase, the internal modeling phase, or the final analyzer handoff.

To systematically verify each phase, the project utilizes the CTest framework to orchestrate four distinct regression suites. Testing was conducted on Linux, specifically Ubuntu 22.04 LTS and Arch Linux. Within the checked-in CMake configuration, the plugin build is parameterized by the `TARGET_GCC` variable, which defaults to `gcc-12`. To verify portability across supported compiler releases, the project was rebuilt from a clean state, and the full `make test` pipeline was rerun separately with GCC 12, GCC 13, and GCC 14. The testing pipeline is fully automated via the `make test` build target, with dedicated entry points for the `cl-adapter`, `codemodel`, `cl_analyze`, and Predator compatibility suites.

5.2 Verification of the Modernized Infrastructure

Before testing the infrastructure against complex third-party tools, the core components of the new architecture were verified in isolation. This ensures that the foundational C++23 implementations, specifically the entity pools, the `std::variant` hierarchies, and the JSON round-trip, function correctly.

This verification is covered by three dedicated test suites:

1. **GCC-Adapter Tests (46 tests):** These tests compile historical C programs through the new live GCC plugin. The generated JSON output is compared against expected

fixtures using shell checks built around `jq` [1]. This suite proves that the Compiler Abstraction Layer correctly intercepts GCC's GIMPLE passes, recovers source locations, and accurately identifies types and control flow edges.

2. **Code Model Integration Tests (38 tests):** These tests compile dedicated C inputs through the live plugin and inspect the resulting JSON model through suite-specific assertion scripts. They verify the representation of primitive and aggregate types, function calls, control flow constructs, initializers, and other edge cases that stress the internal Code Model layout.

3. **Offline Pipeline Tests (3 tests):** These end-to-end tests exercise the standalone `cl_analyze` tool. Each test first exports a Code Model to JSON through the live plugin, then invokes `cl_analyze` with a native analyzer and checks the produced output file. These tests exercise the `JSONFrontend` to `NativeAnalyzerBridge` path using the reference `callgraph_dot` and `recursion_check` analyzers.

All tests across these three suites pass successfully, confirming that the new compiler-independent core is structurally sound.

5.3 Backward Compatibility: Predator Regression Analysis

The most rigorous constraint placed on the modernization effort was ensuring that the VeriFIT group's primary analysis tool, the Predator shape analyzer, continued to function without modifications to its source code. The compatibility patch integrates the Predator GCC regression corpus directly into CTest. In its default fast configuration, this registers 1,192 `new-plugin` tests, corresponding to the GCC regression inputs executed in four modes: the default error-label mode, `track_uninit`, OOM simulation, and `exit_leaks`. These tests cover pointer arithmetic, dynamically linked structures, sorting routines, and memory-management diagnostics [12].

To evaluate backward compatibility, the patched Predator kernel is compiled against the new infrastructure and invoked through the `LegacyPredatorBridge` and `PredatorAdapter`. The harness compares the filtered diagnostic output of the new plugin path against the same expected `.err` files used by Predator's GCC-based regression workflow. Across the clean rebuilds performed for GCC 12, GCC 13, and GCC 14, all 1,192 registered `new-plugin` tests passed successfully.

During development, compatibility debugging was driven repeatedly against this regression harness, with a quick verification subset covering tests 1 through 20 used to track the adapter fixes summarized in the implementation chapter.

The significance of this regression setup lies in its validation criterion. The new object-oriented Code Model is considered compatible only if the reconstructed callback stream reproduces the legacy diagnostic behavior expected by Predator's regression files. This makes the Predator suite the most demanding external check of variable scopes, control-flow reconstruction, and low-level memory-operation fidelity in the current thesis implementation.

[1] https://github.com/jqlang/jq

5.4 Performance Considerations and Architectural Trade-offs

When evaluating the performance of the modernized infrastructure, it is crucial to recognize that the modernization does not merely optimize a single fixed execution path. Instead, it introduces two distinct modes of operation: a native path that analyzes the `CodeModel` directly, and a compatibility path that reconstructs the historical `cl_code_listener` callback stream for Predator. As a result, any direct wall-clock comparison with the legacy framework would combine several different costs, specifically the extraction of GIMPLE, construction of the persistent model, optional annotation requests, adapter replay for legacy tools, and the analyzer's own internal algorithms. In particular, the end-to-end timing of Predator would largely reflect the behavior of its verification kernel, not just the overhead introduced by the new infrastructure.

For this reason, no dedicated benchmarking harness or memory-profiling campaign was implemented as part of this thesis. A trustworthy quantitative study would require a separate experimental methodology with fixed workloads, repeated runs, isolated measurement of the live and offline paths, and a strict separation between framework cost and analyzer-internal cost. Reporting a small set of uncontrolled timing numbers would therefore be less informative than explicitly limiting the claims to what can be justified directly from the implemented architecture and the executed regression campaign.

Structural Overhead (The Double-Translation Cost)

For legacy tools like Predator, the modernized infrastructure introduces a deliberate structural overhead. In the legacy architecture, the GCC plugin translated GIMPLE directly into the `cl_code_listener` callback stream in a single pass. In the new architecture, however, the execution flow performs the work twice: the `GCCAdapter` first translates GIMPLE into the persistent `CodeModel`, allocating entities and resolving IDs, and then the `PredatorAdapter` traverses the entire `CodeModel` to reconstruct and emit the legacy callbacks. Consequently, the baseline structural processing time for legacy tools is inherently higher in the new system. This overhead is specific to the compatibility path and should not be generalized to native analyzers that consume the `CodeModel` directly.

Algorithmic Efficiency (The Elimination of Eager Pre-Analyses)

However, this structural overhead is heavily offset by the elimination of the legacy system's eager pre-analyses. The legacy `ClEasy` pipeline forced the mandatory computation of a points-to graph, loop-closing edges, and variable liveness sets for every analyzed function before the analyzer was even invoked. Because the new infrastructure replaces this eager pipeline with the lazy-evaluating `AnalysisManager`, these expensive algorithms are completely bypassed unless explicitly requested. This design changes where work is paid for: the framework performs more structural translation along the compatibility path, while auxiliary analyses are moved behind an explicit demand-driven boundary. For a fair quantitative comparison, this shift would need to be measured separately from the analyzer's cost, because the two architectures no longer spend time in the same execution phases.

Conclusion on Performance

Ultimately, the modernized infrastructure introduces a clear trade-off. The legacy compatibility path pays for an additional reconstruction step, while the native path operates directly on the `CodeModel` and requests derived analyses only when needed. The strongest conclusion supported by this thesis is therefore not a numeric speedup claim, but a clearer placement of computational cost. The framework now makes the compatibility overhead explicit, removes mandatory framework-side pre-processing from the native path, and preserves correct behavior across the executed regression suites and the tested GCC versions. Quantifying whether this also yields a standalone timing or memory advantage remains future work for a dedicated benchmark campaign.

Chapter 6

Conclusion

This thesis set out to research the original Code Listener infrastructure and to develop a modern replacement, aiming to preserve its static-analysis value while addressing key architectural limitations. To achieve these goals, a new GCC plugin and toolchain were designed around a compiler-independent `CodeModel`, lazy annotation services, and unified interfaces for export and analysis. The resulting prototype enables live analysis inside GCC, offline JSON-based processing through `cl_merge` and `cl_analyze`, and ongoing integration with the existing Predator toolchain. Evaluation showed strong continuity: all native regression suites and the Predator compatibility path passed 1,192 `new-plugin` regression tests during clean rebuilds using GCC 12, 13, and 14. Collectively, these results demonstrate that modernization not only replicates the functionality of the original integration point but also provides a more maintainable and extensible foundation for future analyzers and workflows, thereby guiding subsequent development directions.

Building on these results, the most natural continuation of this work is to expand the framework along the trajectory in which it was originally designed, ensuring cohesive growth. Short-term improvements include introducing the planned `ModelBuilder` abstraction to cleanly separate the incremental GCC extraction logic from the `CodeModel`'s internal storage layout, as well as implementing additional annotation services for analyses currently available only in legacy form, such as loop-closing edges, points-to information, and variable liveness. In the longer term, the architecture could be broadened with another compiler frontend, most notably, an LLVM/Clang adapter, and the offline workflow could be extended to support legacy analyzers without requiring a live GCC execution. Collectively, these directions preserve the main contribution of this thesis: providing a practical compatibility bridge for existing tools and a cleaner platform for future research and engineering.

Bibliography

[1] BoostedCpp. *Inside Boost.Container: Comparing Different Deque Implementations* online. 2026. Available at: https://boostedcpp.net/2026/03/30/deque/. [cit. 2026-04-27].

[2] Dudka, K.; Peringer, P. and Vojnar, T. An Easy to Use Infrastructure for Building Static Analysis Tools. *Lecture Notes in Computer Science*, 2012, vol. 2012, no. 6927, p. 527–534. ISSN 0302-9743. Available at: http://www.springerlink.com/content/75024ol1tk386572/.

[3] Gamma, E.; Helm, R.; Johnson, R. and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. 1st ed. Addison-Wesley, 1994. ISBN 978-0-201-63361-0.

[4] GNU Project. *GNU Compiler Collection Internals* online. Free Software Foundation, 2024. Available at: https://gcc.gnu.org/onlinedocs/gcc-12.5.0/gccint.pdf. [cit. 2026-04-04]. Version 12.5.0.

[5] GNU Project. *C++ Standards Support in GCC* online. 2025. Available at: https://gcc.gnu.org/projects/cxx-status.html. [cit. 2026-04-27].

[6] Kašpar, D. *Extension of the Code Listener Infrastructure Adding C++ Support*. Brno, Czech Republic, 2015. Bachelor's Thesis. Brno University of Technology, Faculty of Information Technology.

[7] Kirchner, F.; Kosmatov, N.; Prevosto, V.; Signoles, J. and Yakobowski, B. Frama-C: A software analysis perspective. *Formal Aspects of Computing*. Springer, May 2015, vol. 27, no. 3, p. 573–609. ISSN 1433-299X. Available at: https://doi.org/10.1007/s00165-014-0326-7.

[8] Lam, P.; Bodden, E.; Lhotak, O. and Hendren, L. The Soot framework for Java program analysis: a retrospective. In: Cetus Users and Compiler Infrastructure Workshop. *Cetus Users and Compiler Infastructure Workshop (CETUS 2011)*. October 2011. Available at: http://tubiblio.ulb.tu-darmstadt.de/59322/.

[9] Lattner, C. and Adve, V. LLVM: A compilation framework for lifelong program analysis & transformation. In: IEEE Computer Society. *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* San Jose, CA, USA: IEEE, 2004, p. 75–86. ISBN 9780769521022. Available at: http://ieeexplore.ieee.org/document/1281665/.

[10] Lohmann, N. *JSON for Modern C++: Header Only* online. 2025. Available at: https://json.nlohmann.me/integration/index.html. [cit. 2026-04-27].

[11] Martin, R. C.; Grenning, J.; Brown, S. and Henney, K. *Clean Architecture: a craftsman's guide to software structure and design*. 1st ed. Boston Columbus Indianapolis New York San Francisco Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo: Prentice Hall, 2018. Robert C. Martin series. ISBN 9780134494166.

[12] Peringer, P.; Šoková, V.; Trtík, M.; Vojnar, T.; Holík, L. et al. Predator Shape Analysis Tool Suite. In: Bloem, R. and Arbel, E., ed. *Proceedings of HVC 2016*. Zurich: Springer International Publishing, 2016, vol. 10028, p. 202–209. Lecture Notes in Computer Science. ISBN 978-3-319-49052-6. Available at: http://link.springer.com/chapter/10.1007/978-3-319-49052-6_13.

[13] Shvets, A. *Large Class: Code Smell* online. 2026. Available at: https://refactoring.guru/smells/large-class. [cit. 2026-04-27].

[14] Tarjan, R. Depth-first search and linear graph algorithms. *SIAM journal on computing*. SIAM, 1972, vol. 1, no. 2, p. 146–160.

[15] VeriFIT Research Group. *Forester* online. 2019. Available at: https://www.fit.vut.cz/research/group/verifit/public/tools/forester/. [cit. 2026-04-27].

[16] VeriFIT Research Group. *Predator* online. 2026. Available at: https://www.fit.vut.cz/research/group/verifit/public/tools/predator/. [cit. 2026-04-27].